java garbage collection interview questions

What is garbage collection in Java?

Answer:

- Garbage Collection is an automated process in Java that reclaims memory occupied by objects no longer referenced, ensuring efficient memory management and preventing memory leaks.

What is the role of the System.gc() and Runtime.gc() methods?

Answer:

- Both methods are requests to the JVM to initiate garbage collection, but they do not guarantee that GC will run immediately. The JVM decides when to perform GC.

Can you force garbage collection in Java?

Answer:

- No, you cannot force garbage collection. You can only suggest it by calling System.gc() or Runtime.gc(). The JVM controls when and how GC is executed.

What is a memory leak in Java?

Answer:

- A memory leak occurs when objects are no longer needed but are not garbage collected because references to them are still held.

What is the significance of the finalize() method?

Answer:

- The finalize() method is called by the garbage collector on an object before it is destroyed to allow resource cleanup.
- The finalize() method is deprecated in Java 9 and removed in Java 18 due to reliability and performance issues.

What are strong, weak, soft, and phantom references?

Answer:

- Strong Reference: The default type of reference. If an object has a strong reference, it won't be garbage collected.
- Weak Reference: Objects are garbage collected when they are weakly reachable (used in caching).
- Soft Reference: Objects are garbage collected only when the JVM is low on memory (used in memory-sensitive caches).
- Phantom Reference: Used for post-mortem cleanup; an object is added to a ReferenceQueue after being finalized.

What are the different types of garbage collectors in Java?

Answer:

- Serial GC: Single-threaded, suitable for small applications.
- Parallel GC: Multi-threaded, for throughput-focused applications.
- CMS (Concurrent Mark-Sweep) GC: Low-latency, deprecated in Java 9.
- G1 (Garbage First) GC: Balanced between throughput and low latency.
- ZGC (Z Garbage Collector): Handles very large heaps with low latency (introduced in Java 11).
- Shenandoah GC: Low pause times (introduced in Java 12).

How does the G1 garbage collector work?

Answer:

- G1 GC divides the heap into regions and prioritizes garbage collection in regions with the most garbage (hence " Garbage First ").
- It balances throughput and latency by reducing stop-the-world pauses.

What are the phases of garbage collection in Java?

Answer:

- **Mark**: Identify which objects are still reachable.
- **Sweep**: Remove unreachable objects and reclaim memory.
- **Compact**: Rearrange objects to reduce fragmentation (used by some collectors).

What is heap fragmentation, and how is it handled?

Answer:

- Heap fragmentation occurs when free memory is scattered across the heap in non-contiguous blocks. It is handled by compacting objects during garbage collection.

What happens if the heap is full and garbage collection cannot free up space?

Answer:

- If the heap is full and GC cannot reclaim enough memory, the JVM throws an OutOfMemoryError.

How do you prevent memory leaks in Java?

Answer:

- Use weak references for cache-like structures.
- Remove listeners, callbacks, or references when objects are no longer needed.
- Avoid static references to objects.
- Use tools like Eclipse MAT to detect memory leaks.

What are the different regions of the heap?

Answer:

- Young Generation:

Divided into Eden and Survivor Spaces.

Objects are created in Eden and moved to Survivor spaces if they survive garbage collection.

- Old Generation (Tenured):

Stores long-lived objects that survived multiple GC cycles in the Young Generation.

- Metaspace:
- Stores class metadata and is separate from the heap.

What is stop-the-world (STW) in garbage collection?

Answer:

- STW refers to the JVM pausing all application threads during certain phases of garbage collection, which can impact application performance.